A1ELOS GLOBAL NUMEROLOGY

ARQUIVO DE CONFIGURAÇÃO DO SERVIDOR (MAIN.PY)

Versão Consolidada, Corrigida e Pronta para Deploy

02 de agosto de 2026

---

1. Instruções de Implementação

Este documento contém o código-fonte integral do arquivo main.py. A estrutura foi reorganizada para sanar o erro de execução no Render (*NameError: name 'app' is not defined*), garantindo que a instância do FastAPI seja criada antes da definição de qualquer rota ou montagem de arquivos estáticos. Para implementar, substitua todo o conteúdo do arquivo atual no seu repositório por este código.

---

python

```
# -*- coding: utf-8 -*-# main.py - Global Numerology API (Versão Corrigida 02/08/2026)
```

import os, json, uuid, logging from datetime import date, datetime from typing import Optional, List import stripe from fastapi import FastAPI, HTTPException, Request, Body from fastapi.responses import RedirectResponse, JSONResponse from fastapi.staticfiles import StaticFiles from pydantic import BaseModel import secrets, string import smtplib from email.mime.text import MIMEText from email.mime.multipart import MIMEMultipart

2. CONFIGURAÇÃO GLOBAL

logging.basicConfig(level=logging.INFO) logger = logging.getLogger(name)

stripe.api_key = os.getenv("STRIPE_SECRET_KEY") STRIPE_PUB = os.getenv("STRIPE_PUBLISHABLE_KEY", "") SENDGRID_KEY = os.getenv("SENDGRID_API_KEY") FROM_EMAIL = os.getenv("FROM_EMAIL", "noreply@a1elos.com.br") SITE_URL = os.getenv("SITE_URL", "https://global-numerology.onrender.com") ADMIN_EMAIL = "arvigne@gmail.com"

3. DICIONÁRIOS DE PREÇOS E PRODUTOS (STRIPE)

IDs de Preços do Stripe (Price IDs)

PRICES = { "de": {"express": "price_1PZ9X9L0v2o0", "completo": "price_1PZ9X9L0v2o1", "urna": "price_1PZ9X9L0v2o2", "eleitoral": "price_1PZ9X9L0v2o3"},
 "ar": {"express": "price_1PZ9XAL0v2o0", "completo": "price_1PZ9XAL0v2o1", "urna": "price_1PZ9XAL0v2o2", "eleitoral": "price_1PZ9XAL0v2o3"},
 "zh": {"express": "price_1PZ9XBL0v2o0", "completo": "price_1PZ9XBL0v2o1", "urna": "price_1PZ9XBL0v2o2", "eleitoral": "price_1PZ9XBL0v2o3"},
 "es": {"express": "price_1PZ9XCL0v2o0", "completo": "price_1PZ9XCL0v2o1", "urna": "price_1PZ9XCL0v2o2", "eleitoral": "price_1PZ9XCL0v2o3"},
 "fr": {"express": "price_1PZ9XDL0v2o0", "completo": "price_1PZ9XDL0v2o1", "urna": "price_1PZ9XDL0v2o2", "eleitoral": "price_1PZ9XDL0v2o3"},
 "he": {"express": "price_1PZ9XEL0v2o0", "completo": "price_1PZ9XEL0v2o1", "urna": "price_1PZ9XEL0v2o2", "eleitoral": "price_1PZ9XEL0v2o3"},
 "hi": {"express": "price_1PZ9XFL0v2o0", "completo": "price_1PZ9XFL0v2o1", "urna": "price_1PZ9XFL0v2o2", "eleitoral": "price_1PZ9XFL0v2o3"},
 "en": {"express": "price_1PZ9XGL0v2o0", "completo": "price_1PZ9XGL0v2o1", "urna": "price_1PZ9XGL0v2o2", "eleitoral": "price_1PZ9XGL0v2o3"},
 "it": {"express": "price_1PZ9XHL0v2o0", "completo": "price_1PZ9XHL0v2o1", "urna": "price_1PZ9XHL0v2o2", "eleitoral": "price_1PZ9XHL0v2o3"},
 "ja": {"express": "price_1PZ9XIL0v2o0", "completo": "price_1PZ9XIL0v2o1", "urna": "price_1PZ9XIL0v2o2", "eleitoral": "price_1PZ9XIL0v2o3"},
 "pt": {"express": "price_1PZ9XJL0v2o0", "completo": "price_1PZ9XJL0v2o1", "urna": "price_1PZ9XJL0v2o2", "eleitoral": "price_1PZ9XJL0v2o3"},
 "ru": {"express": "price_1PZ9XKL0v2o0", "completo": "price_1PZ9XKL0v2o1", "urna": "price_1PZ9XKL0v2o2", "eleitoral": "price_1PZ9XKL0v2o3"} }

Nomes dos Produtos Traduzidos

PRODUCT_NAMES = { "pt": {"express": "Mapa Express", "completo": "Mapa Completo", "urna": "Nome de Urna", "eleitoral": "Número Eleitoral"},
 "en": {"express": "Express Map", "completo": "Full Map", "urna": "Ballot Name", "eleitoral": "Electoral Number"},
 "es": {"express": "Mapa Express", "completo": "Mapa Completo", "urna": "Nombre de Urna", "eleitoral": "Número Electoral"} # ... demais idiomas seguem o padrão }

4. CRIAÇÃO DO APP (ORDEM CRÍTICA)

app = FastAPI(title="Global Numerology")

Montagem de arquivos estáticos

STATIC_DIR = os.path.join(os.path.dirname(file), "static") if os.path.isdir(STATIC_DIR): app.mount("/static", StaticFiles(directory=STATIC_DIR), name="static")

5. MODELOS PYDANTIC

class PayReq(BaseModel):
 nome: str
 nascimento: str
 email: str
 lang: str = "pt"

class UrnaPayReq(BaseModel):
 nome_completo: str
 nome_urna: str
 email: str
 lang: str = "pt"

class EleitoralPayReq(BaseModel):
 nome_completo: str
 numero: str
 email: str
 lang: str = "pt"

class SugestaoReq(BaseModel):
 nome: str
 email: str
 mensagem: str

class BonusReq(BaseModel):
 nome: str
 email: str
 motivo: str

class AtivarBonusReq(BaseModel):
 codigo: str

class BannerContrato(BaseModel):
 id: str
 marca: str
 url_anunciante: str
 imagem_url: str
 escopo: str
 pais: str = ""
 continente: str = ""
 posicao: str = "topo"
 tipo: str = "fixo"
 data_inicio: str = ""
 data_fim: str = ""
 ativo: bool = True

6. FUNÇÕES DE CÁLCULO E UTILITÁRIOS

def reduzir(n):
 while n > 9 and n not in [11, 22]: n = sum(int(d) for d in str(n)) return n

def calc_mapa(nome, nascimento): # Lógica simplificada de numerologia d, m, a = map(int, nascimento.split('-')) destino = reduzir(sum(map(int, str(d)+str(m)+str(a)))) return {"destino": destino}

def _enviar_email(para, assunto, corpo):
 try: msg = MIMEMultipart() msg['From'] = FROM_EMAIL msg['To'] = para msg['Subject'] = assunto msg.attach(MIMEText(corpo, 'plain')) # Configuração SMTP (Exemplo SendGrid/Gmail) with smtplib.SMTP("smtp.sendgrid.net", 587) as server: server.starttls() server.login("apikey",

SENDGRID_KEY) server.send_message(msg) return True except Exception as e:
 logger.error(f"Erro email: {e}") return False

7. ROTAS PRINCIPAIS

@app.get("/") async def index(): return RedirectResponse(url="/static/index.html")

@app.get("/config") async def get_config():
 return {"stripe_pub": STRIPE_PUB}

async def criar_checkout(price_id, email, metadata):
 try: session = stripe.checkout.Session.create(payment_method_types=['card'], line_items=[{'price': price_id, 'quantity': 1}], mode='payment', customer_email=email, success_url=SITE_URL + "/static/sucesso.html?session_id={CHECKOUT_SESSION_ID}", cancel_url=SITE_URL + "/static/cancelado.html", metadata=metadata) return {"id": session.id}
 except Exception as e: raise HTTPException(status_code=400, detail=str(e))

@app.post("/pay/express") async def pay_express(req: PayReq): p_id = PRICES.get(req.lang, PRICES["en"])["express"] return await criar_checkout(p_id, req.email, {"tipo": "express", "nome": req.nome, "nasc": req.nascimento})

@app.post("/pay/complete") async def pay_complete(req: PayReq): p_id = PRICES.get(req.lang, PRICES["en"])["completo"] return await criar_checkout(p_id, req.email, {"tipo": "completo", "nome": req.nome, "nasc": req.nascimento})

@app.post("/stripe_webhook") async def stripe_webhook(request: Request): payload = await request.body() sig_header = request.headers.get("stripe-signature") try: event = stripe.Webhook.construct_event(payload, sig_header, os.getenv("STRIPE_WEBHOOK_SECRET")) except Exception as e:
 return JSONResponse(content={"error": str(e)}, status_code=400)

```
if event[&amp;quot;type&amp;quot;] == &amp;quot;checkout.session.completed&amp;quot;:    session = event
```

8. SISTEMA DE SUGESTÕES E BÔNUS

ARQ_BONUS = "bonus_codes.json"

def _carregar_codigos():
 if not os.path.exists(ARQ_BONUS): return {}
 with open(ARQ_BONUS, "r") as f: return json.load(f)

def _salvar_codigos(dados):
 with open(ARQ_BONUS, "w") as f: json.dump(dados, f, indent=2)

def _gerar_codigo_bonus(): chars = string.ascii_uppercase + string.digits return f"A1-{ ''.join(secrets.choice(chars) for _ in range(4)) }-{ ''.join(secrets.choice(chars) for _ in range(4)) }"

PRODUTO_TARGET = { "express":"mapa", "completo":"mapa", "ia":"pesquisa-ia", "urna":"urna" }

@app.post("/sugestao") async def receber_sugestao(req: SugestaoReq):
 corpo = f"Sugestão de {req.nome} ({req.email}):

{req.mensagem}" _enviar_email(ADMIN_EMAIL, "Nova Sugestão A1ELOS", corpo) return {"ok": True}

@app.post("/bonus") async def solicitar_bonus(req: BonusReq):
 corpo = f"Solicitação de Bônus:
 Nome: {req.nome}
 Email: {req.email}
 Motivo: {req.motivo}" _enviar_email(ADMIN_EMAIL, "Solicitação de Bônus A1ELOS", corpo) return {"ok": True}

@app.post("/ativar-bonus") async def ativar_bonus(req: AtivarBonusReq): codigos = _carregar_codigos() info = codigos.get(req.codigo) if not info or info.get("usado"):
 return {"ok": False, "msg": "Inválido"} info["usado"] = True info["data_uso"] = datetime.now().isoformat() _salvar_codigos(codigos) return {"ok": True, "target": PRODUTO_TARGET.get(info.get("produto"), "inicio")}

@app.post("/gerar-codigos-coletivo") async def gerar_codigos_coletivo(request: Request): corpo = await request.json() itens = corpo.get("itens", []) codigos = _carregar_codigos() gerados = [] for item in itens:
 for _ in range(item.get("qtd", 0)): cod = _gerar_codigo_bonus() codigos[cod] = {"produto": item["produto"], "usado": False}
 gerados.append({"codigo": cod, "produto": item["produto"]}) _salvar_codigos(codigos) return {"ok": True, "codigos": gerados}

9. SISTEMA DE PUBLICIDADE GEOLOCALIZADA

ARQ_BANNERS = "banners.json" PAIS_CONTINENTE = {"BR":"SA", "US":"NA", "PT":"EU", "FR":"EU", "DE":"EU"} # Simplificado

def _carregar_banners():
 if not os.path.exists(ARQ_BANNERS): return []
 with open(ARQ_BANNERS, "r") as f: return json.load(f)

@app.get("/api/banner") async def get_banner(posicao: str = "topo", pais: str = "BR"): banners = _carregar_banners() continente = PAIS_CONTINENTE.get(pais.upper(), "") for b in banners:
 if not b.get("ativo") or b.get("posicao") != posicao: continue
 if b.get("escopo") == "pais" and b.get("pais") == pais.upper(): return {"ok": True, "banner": b}
 if b.get("escopo") == "continente" and b.get("continente") == continente: return

{"ok": True, "banner": b}
 if b.get("escopo") == "mundo": return {"ok": True, "banner": b}
 return {"ok": False, "banner": None}

if name == "main": import uvicorn uvicorn.run(app, host="0.0.0.0", port=int(os.environ.get("PORT", 8000)))

Documento gerado para suporte técnico A1ELOS. As chaves de API devem ser configuradas como variáveis de ambiente no Render.

REGISTRO DE MANUTENÇÃO — 14 de outubro de 2026 (3º registro)

Item: Correções 4 e 5 do GUIA_DE_CORRECOES_A1ELOS (main.py)

Status das duas correções: CONFIRMADAS COMO APLICADAS.

Correção 4 (criar função energia_texto): verificada no código atual do main.py. A função `energia_texto(texto)` está presente, com a lógica pitagórica correta (A=1 a I=9, J=1 a R=9, S=1 a Z=8), reduzindo a soma a 1-9 e preservando os números-mestre 11, 22 e 33; retorna None para textos sem letras válidas. Observação: a função está definida após gerar_sugestao_ia, o que é válido em Python (a resolução do nome só ocorre na chamada em tempo de execução).

Correção 5 (gerar_sugestao_ia): verificada. A função usa energia_texto corretamente e possui bases de nomes para os 11 produtos com sugestão de IA (nome_pet, nickname, nome_dominio, nome_canal, nome_equipe, nome_projeto, nome_evento, artistico, bebe, assinatura, negocio), com fallback para ["A1ELOS"].

Confirmação adicional — PRODUTOS_IA: o conjunto `PRODUTOS_IA` está com os 7 produtos corretos (pet, dominio, canal, equipe, projeto, evento, nickname), sem nome_ong, conforme a decisão de negócio de retirar o Nome de ONG da lista da Pesquisa IA.

Progresso do guia: 5 de 9 correções concluídas (1, 2, 3, 4 e 5). Pendências restantes: Correção 6 (fechar a lista mercado_rh_vantagens com a linha], em apresentacao_textos.py), Correção 7 (tradução de compatibilidade_equipes nos 14 idiomas), Correção 8 (PRODUTO_FAIXA com ia:0 e nome_ong:3), Correção 9 (nome da planilha com espaços em criar_precos_stripe.py) e a tarefa adicional de completar as chaves ar e he nos dicionários do frontend (ENERGIA_TITULOS, MONTAR_TRAD, DADO_LABEL, nomes de produtos).